

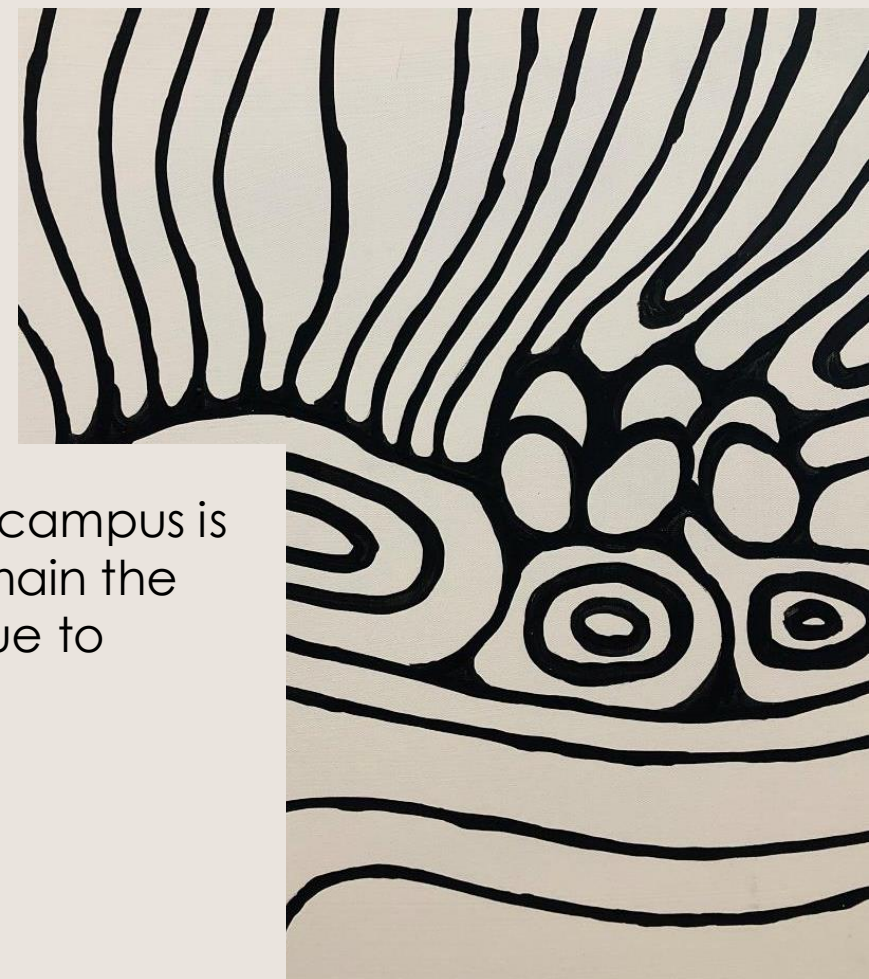


Topic One: Introduction

BUSN5101: Programming for Business

Acknowledgement of country

The University of Western Australia acknowledges that its campus is situated on Noongar land, and that Noongar people remain the spiritual and cultural custodians of their land, and continue to practise their values, languages, beliefs and knowledge.



Artist: Dr Richard Barry Walley OAM

New to UWA? Let us help you settle in.



Get your
'Just For Me, How
To UWA Guide'

All you have to do is answer a few questions.
You'll get a personalised list of tips, resources and support services
sent direct to your student email.



UWA
PERTH · AUSTRALIA

Are you a new Student?

Complete CARS early

Available in the LMS

Communication and Research Skills (CARS) is an online unit that aims to assist students at UWA to develop communication and research skills in an academic context.



THE UNIVERSITY OF
**WESTERN
AUSTRALIA**

Have you enrolled in a new course?

Complete ACE early

If ACE is not completed,
an encumbrance will be applied to your student record

Available in the LMS

Academic Conduct

Essentials (ACE) is a compulsory online module for all students about ethical scholarship and the expectations of correct academic conduct that UWA has of its students.



THE UNIVERSITY OF
**WESTERN
AUSTRALIA**

Your Student Wellbeing Hub



visit Shenton House



The Living Room

Drop in and chat with a Peer Supporter and enjoy the dog visits Monday to Friday, 11am-4pm

International Student Lounge

Drop in for dedicated help with settling into Perth and UWA and make new friends

Support for sexual harm and family and domestic violence

Access confidential support from a specialist about safety, academic, wellbeing and reporting options

Volunteer Opportunities

Meet people with similar interests, gain confidence, add skills and experiences to your CV. Drop in to find out more

Pop-up events and workshops

Weekly wellbeing activities throughout semester – stay tuned on UWA student socials

Your Support Services



uwa.edu.au/students/support-services

bookmark your
services
directory

Academic skills

Get advice, one-on-one support and workshops to improve your skills in academic writing, study, English language, research, maths and stats

Accessibility support

Is a physical or mental health condition affecting your studies? Book online for an appointment to explore learning adjustments with UniAccess

Counselling

Book online for a free counselling appointment

Medical Centre

Book online to see a GP (doctor). Bulk billed for domestic, OSHC covered for international students. Medibank is our preferred provider

Security

24/7 assistance and security escorts to car parks and College Row. Download the Safe Zone App

**55% of UWA
students don't
know where to go
for support for
sexual harm.**

Let's change that.

Respectful Relationships: let's
get on the same page



After completing Respectful Relationships 88% of students feel more confident to refer a friend to appropriate support services

Do the Respectful Relationships module today on LMS



PROCEDURE FOR SAFETY INSTRUCTION TO STUDENTS IN UWA BUSINESS SCHOOL

The Business School requires that **all** students receive instruction on emergency and evacuation procedures for all teaching venues within the building at the beginning of the third lectures/tutorials attended by students. This instruction is important due to the size and design of the building and also the number of people using the building.

This instruction is the responsibility of the unit coordinator/tutor and the following procedure should be performed and confirmation of the instructions will be documented.

The person delivering the introductory lecture is responsible for ensuring their audience receives the correct details for the specific teaching venue. The person in charge of the class shall act as the Warden and is responsible for evacuating the area.

The following instruction on the emergency evacuation procedure is required:

On hearing the evacuation alarm:

- Immediately cease all activities and collect hand-held belongings
- Turn off any electrical devices
- In controlled sequence, move along and exit in an orderly manner through nearest exits
- Walk quickly and calmly as directed to the designated assembly area at the **centre** of the Human Movement Oval
- Remain at the assembly area until otherwise instructed

No person is to re-enter the building until the Building Warden gives the All Clear

Also please advise students of the following:

1. They should attend lectures and tutorials only at their designated times to ensure venues are not overcrowded.
2. The University is a smoke free campus. Smoking is prohibited in, or at, all of the University's buildings, properties and workplaces.
3. Speed limit in the UWABS basement car park is 15kph to ensure the safety of users and your car.
PLEASE NOTE: The basement car park is for Staff only during business hours, 8am to 5pm.
4. UWA Emergency Telephone Number is 6488 2222; put this number in your phone now!

Welcome



- This is Programming in Business (BUSN5101) at the University of WA Business School;
- My name is Tristan W. Reed (please call me Tristan) and I am the Unit Coordinator and Lecturer for this unit as well as the Tutor for some of the seminars (laboratories);
- Additional teaching staff will be joining us for the rest of the seminars (labs);
- The best way to contact me is via email in the first instance to tristan.reed@uwa.edu.au;
- **If you aren't enrolled (and aren't waiting to) in BUSN5101, you've come to the wrong place or come at the wrong time – check CAS to see where you should be!**

Unit Outcomes

(Per the handbook and outline, on successful completion of this unit) students are able to:

1. Demonstrate an understanding of basic types of algorithms and the benefits of programs for automating various tasks;
2. Validate programs, recognise how to avoid common coding errors, and perform individual and team program reviews;
3. Evaluate fundamental data structures and associated algorithms;
4. Create and evaluate algorithms for solving business problems; and
5. Evaluate basic technical documents, presentations and group interactions, using appropriate tools.

Unit Layout

- Each week, we will have a *one-hour lecture* (you're watching now) as well as a *two-hour seminar* (i.e. computer lab – you should be enrolled and generally attend only one – these start Week 2);
- We will have *one individual assignment*, *one group assignment* and a *final exam* throughout the semester – more info later on when these are due;
- We will be using LMS for all of our work (submission and notes) and email for communication (we are not using Microsoft Teams).

Week	Topic
1	Introduction
2	Data Types & Variables
3	Data Structures
4	Control Flow (IF/CASE)
5	Looping (FOR/WHILE)
6	User-Defined Functions
7	Python Internal Modules
8	Python External Modules
9	Data Analysis
10	Data Visualisation
11	File Input & Output (I/O)
12	Object Orientation



Topic One: Trends

BUSN5101: Programming for Business

Digital Transformation Trends

Top Strategic Technology Trends 2024

- 1 AI Trust, Risk and Security Management
- 2 Continuous Threat Exposure Management
- 3 Sustainable Technology
- 4 Platform Engineering
- 5 AI-Augmented Development
- 6 Industry Cloud Platforms
- 7 Intelligent Applications
- 8 Democratized Generative AI
- 9 Augmented Connected Workforce
- 10 Machine Customers

Source: Gartner
© 2023 Gartner, Inc. and/or its affiliates. All rights reserved. CM_GTS_2080051

Gartner

IT trends to watch

1

Artificial intelligence

AI is about machines with human attributes - speaking, reading, seeing and even recognising emotion - completing tasks while also "learning" from repeated interactions. Using algorithms that adapt to location, speech or user-history machines can perform tasks that are dangerous or tedious, more accurately or much faster than humans.

Within a few years, analysts predict that all software will use AI at some level, according to US research and advisory firm Gartner. Importantly AI offers the opportunity to continuously tailor products and services providing a competitive advantage over rivals that is not easily copied. The question to ask is 'how can AI help my organisation?'

Augmented reality

Systems that combine real-time 3D vision, sound, haptics (the sense of touch), location data and even other senses such as smell enable people to immerse themselves somewhere else, react to what's around them and alter their virtual environment in real-time. Organisations are increasingly applying this technology across a wide spectrum of human activity from art and entertainment to commerce, education and the military. It's used to train doctors, nurses, teachers and police officers and will soon be available on your smart device. Could you use AR to lift efficiency for your internal stakeholders or help you communicate with your customer base?

3

Blockchain

The fortunes of digital currency Bitcoin have drawn public attention to Blockchain technology, but this secure system for recording and verifying transactions and storing trusted records has the potential to disrupt enterprises of many kinds. Companies are using Blockchain technology to transform time-consuming, centralised, less reliable and less secure systems. Digital democracy platform MyVote, for example, uses Blockchain to store users' personal data and voting history to give citizens a more direct voice in the political process. Could you use Blockchain to keep your data secure?

Automation

Robots in manufacturing go back to the 1960s. Now it's the scale and breadth of the transformation that automated systems make possible, as a result of other advances in machine learning and connectivity, for example, that puts automation firmly at the forefront of technology trends. From convenient devices at home to industrial applications on a massive scale, automation will be a key focus of technological change, with potentially far-reaching economic and social consequences.

Currently, professional services such as the legal and finance industry are being disrupted by automation with feedback from these sectors being that core technical skills, together with management and people skills are more important than ever. How will automation disrupt your industry?

4

Internet of Things

Gartner calls the combination of technologies and the connection of people, devices, content and services the "intelligent digital mesh." This is the foundation for new business models, platforms and possibilities that will transform how we live and work with implications that go far beyond the technology itself and involve disciplines such as law, economics, business and politics.

It is early days for the application of IoT strategy, but it is clear that opportunities will exist for those with the technical knowledge to connect platforms as well as those with the data analytics skills to utilise the rich stream of information generated by IoT applications. What could you learn from connecting and analysing the data from your products or elements of your core operations?

5

The Need for Programming in Biz

- A variety of the upcoming / current trends identified by Gartner and/or UTS require programming and data analysis skills (among others) to deliver and increase business value – of recent relevance and note, AI, as well as applications and platforms;
- Analytical complexity is increasing as the type of data itself increases in complexity, scope and volume – more observations, more variables, more complex techniques;
- The programming skills we will learn throughout this unit will enable us to solve both complex and simple business problems – there is value in both.
- **Have a think – have you undertaken programming before? If so, how? What programming languages have you used, what kind of things have you built?**



Topic One: Processes

BUSN5101: Programming for Business

Process of Programming

- *What is programming?* The process of designing and implementing a sequence of instructions to enable a computer to accomplish a particular task.
- *How is it achieved?* There are many processes we can undertake to do this, however we will use a four step process as described below:
 - *Problem Definition:* understand *what* we are trying to solve – what is the information that we have and what the information is that we will need to create to solve the problem;
 - *Algorithm Design:* design a solution to the problem – that is, take the information that we have and transform it (over one or more steps) to the information that we need to create – *then evaluating it!*;
 - *Program Implementation:* implement the solution using a programming language to solve the problem;
 - *Testing and Evaluation:* ensure the program works correctly and efficiently, repeating the steps above if needed when issues are found with the code.

Process of Programming (II)

- The third step is often known as ‘coding’ as it is where we write the program source code to solve the problem, however it should not be the only step we take;
 - **Think – why do we think that we need to undertake the other steps?**
- The design of our algorithm and most computer processes in general can be broken down into the three steps detailed below:
 - *Input*: gather, input, load and store the information we need to solve the problem;
 - *Process*: analyse, change, modify, transform and/or evaluate the data that was input;
 - *Output*: display, store or output the result of the above input and processing steps;
- What we do for each of these steps will differ depending on the *Problem Definition*.

- In simple terms, an algorithm can be considered as *a defined set of steps required to unambiguously solve a problem*. There are many other (similar) definitions, as well;
 - For those interested in algorithms, Cormen *et al.*'s *Introduction to Algorithms* is an influential book in this area and a great resource – a personal favourite – however, its scope is *far beyond* what is required for this unit;
 - The above definition would include non-computing procedures and only implies a sequential nature;
- The Persian mathematician al-Khwarizmi (known as the “father of algebra”) introduced the modern methods and concepts that formed the basis of what an algorithm is;
 - He built upon mathematical methods from antiquity (‘ancient times’) throughout the world;
 - Modern history of ‘machine algorithms’ strongly influenced by the ground-breaking work of Turing, Lovelace and many others – with work continuing to this day to push the theoretical foundation forwards;

Good Algorithms

- We should strive to (but not always achieve) the following features in an algorithm:
 - *Precision*: the algorithm solves the whole problem at hand – not too broadly or narrowly;
 - *Uniqueness*: if the problem is already solved, then we may wish to use an existing algorithm;
 - *Feasibility*: the steps stated should be able to logically and practically be undertaken so it can be done;
 - *Input and Output*: we know these are (generally) the building blocks of our programming problems;
 - *Finiteness*: the program can execute and finish – not be infinite in nature or impracticably slow to run;
 - *Generality*: the ability to generalise and use the algorithm to solve other similar problems (i.e. different input data with the same format) with minimal (or no) changes;

Algorithm Development

- *There is no single way to write an algorithm* – each one will differ depending on what the problem is you're trying to solve, what resources you have and your own style;
- *Algorithms are language agnostic* – the idea being that you *implement* the algorithm in your language of choice (or whatever has been specified) using instructions that have been specified in plain English but still using the same general programming constructs;
- Algorithms shouldn't be written until we have undertaken the *Problem Definition* stage of programming development – how can we solve a problem when we don't know what the problem is? (*Hint: not very well!*)

Stages of Algorithm Development

- Recall the steps of programming from our previous slides:
 - *Problem Definition, Algorithm Design, Program Implementation, Testing & Evaluation.*
- The *Algorithm Design* step can be further broken down into sub-steps:
 - Modelling, Design, Specification, Checking, Analysis.
- The above steps help us design the algorithm by helping us consider additional requirements (in terms of methodology and design).
- We strive to ensure *efficiency* of our algorithm at run-time (measured using Big-O notation – order of magnitude of run time).

Specification of Algorithms

- Algorithms can be specified (or expressed) in many ways – two of the most popular ways are *Pseudocode* and *Flowcharts*.
 - *Pseudocode*: specifying the steps required using English statements, similar to a recipe for cooking something or a method / set of steps to fix a car or something around the house;
 - *Flowcharts*: a diagram using a standardised set of shapes and lines / arrows to specify how data 'flows' through the process and the paths that it can take;
- One way is not better than the other, but we will be using Pseudocode in this unit – as it is very complimentary to how we are learning program implementation (Python).

Pseudocode and Flowcharts

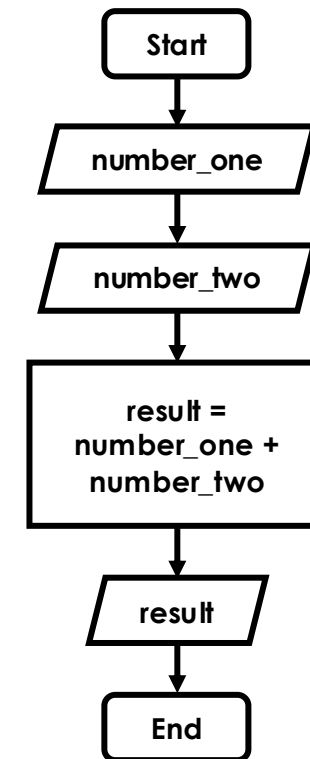
Problem: Ask the user to enter in two numbers, add them together and display the output.

Very
Simple
Example

Pseudocode (below)

```
START
INPUT number_one
INPUT number_two
result <- number_one + number_two
OUTPUT result
END
```

Flowchart (on right)





Topic One: Basics

BUSN5101: Programming for Business

Quick Exercise (pause and try)

Using Pseudocode, write an algorithm that asks the user to enter three numbers, subtracts the second from the first and multiplies the result by the third.

Programming Languages

- Programming languages allow us to *implement* our algorithm using *source code* that is processed by a computer, which can then be executed (code runs so we can use it);
- A programming language is a set of syntax (commands/instructions) with a specific notation used for the specification of an algorithm – many of these exist;
- Programming languages can be broken down into:
 - high level languages (such as Java and Python using English-like statements to make them easy for humans to understand) and;
 - low level languages (such as Assembly or Machine Code which uses binary or mnemonics which are harder to understand).

Compiled vs Interpreted

- Programming languages can also be broken down / delineated into two groups based upon whether they are *compiled* or *interpreted* for execution.
 - *Compiled languages* must first transform the source code into machine code, generating a new file, which can then be run by the computer (a change in the source code would require re-compilation);
 - *Interpreted languages* use a program called an interpreter at run-time to transform the source code instructions into machine code which can be run by the computer;
- There are advantages and disadvantages to both of these approaches, but the language generally dictates which approach is taken:
 - Compiled languages are generally faster than interpreted ones – but these days, not always;
 - Interpreters don't need to generate intermediate files – less work and a more tidy file system;
 - The 'workflow' of execution differs by type (how you go about it, memory usage, error checking, etc.)

Programming Paradigms

- A *paradigm* can be considered a model used to solve a problem – a set of patterns, features and (to a degree) frameworks used. Common ones are:
 - *Procedural*: instructions are executed one after another, 'generally' top to bottom;
 - *Object Oriented*: entities ('things') are modelled as Classes, Objects are instances of them;
 - *Functional*: everything can be considered a mathematical function, which is evaluated;
 - *Logical*: programs are composed of logical statements that express facts i.e. 'rules'.
- The number of existent programming languages are almost too numerous to count.
 - New ones are created for – or optimised for – a specific task (scientific computing, web, databases) or paradigm or to use a different syntax or whatever other reason exists.

Python Programming Language

- Python is an interpreted, high-level and general-purpose programming language;
 - Created in the late 1980's by Guido van Rossum and continually updated since then;
- It is used in many real-world business applications around the world for a variety of purposes - including data analysis, web applications, machine learning and more;
 - Think of your favourite social media platform – it likely does or has used Python (such as Instagram);
- For many reasons of its design, it is a great starting point for beginners:
 - *Easy to Use*: straightforward, English-like syntax with ~30 basic keywords;
 - *Powerful*: simple and (relatively) efficient for most tasks, has dynamic typing (use debatable);
 - *Object Orientation*: supports modularisation and classes for a modern design paradigm of problem solving;
 - *Open Source*: anyone can download or improve it, lowers costs and improves (community) support;

Python Programming Language (II)

- For many reasons of its design, it is a great starting point for beginners:
 - *Integration*: can be easily integrated with other programming languages (e.g. using Cython and Jython);
 - *Platform Independent*: available for development and use on almost any computer architecture;
 - *Libraries and Frameworks*: massive ecosystem of additional tools / code that we can integrate with our own code and tools to make this very easy to do;
- The language was named by van Rossum after the British comedy series *Monty Python's Flying Circus*; the impact of this can be seen throughout the Python ecosystem;
 - One program used to write Python code is known as IDLE – likely somewhat after the comedy Eric Idle;
- While conceptualised in the 1980's, the first version of Python was released in 1991, the second (Python 2) version in 2000 and the third version (Python 3) in 2008;
 - As Python 2 support ended in 2020, we will be using the still-updated Python 3.



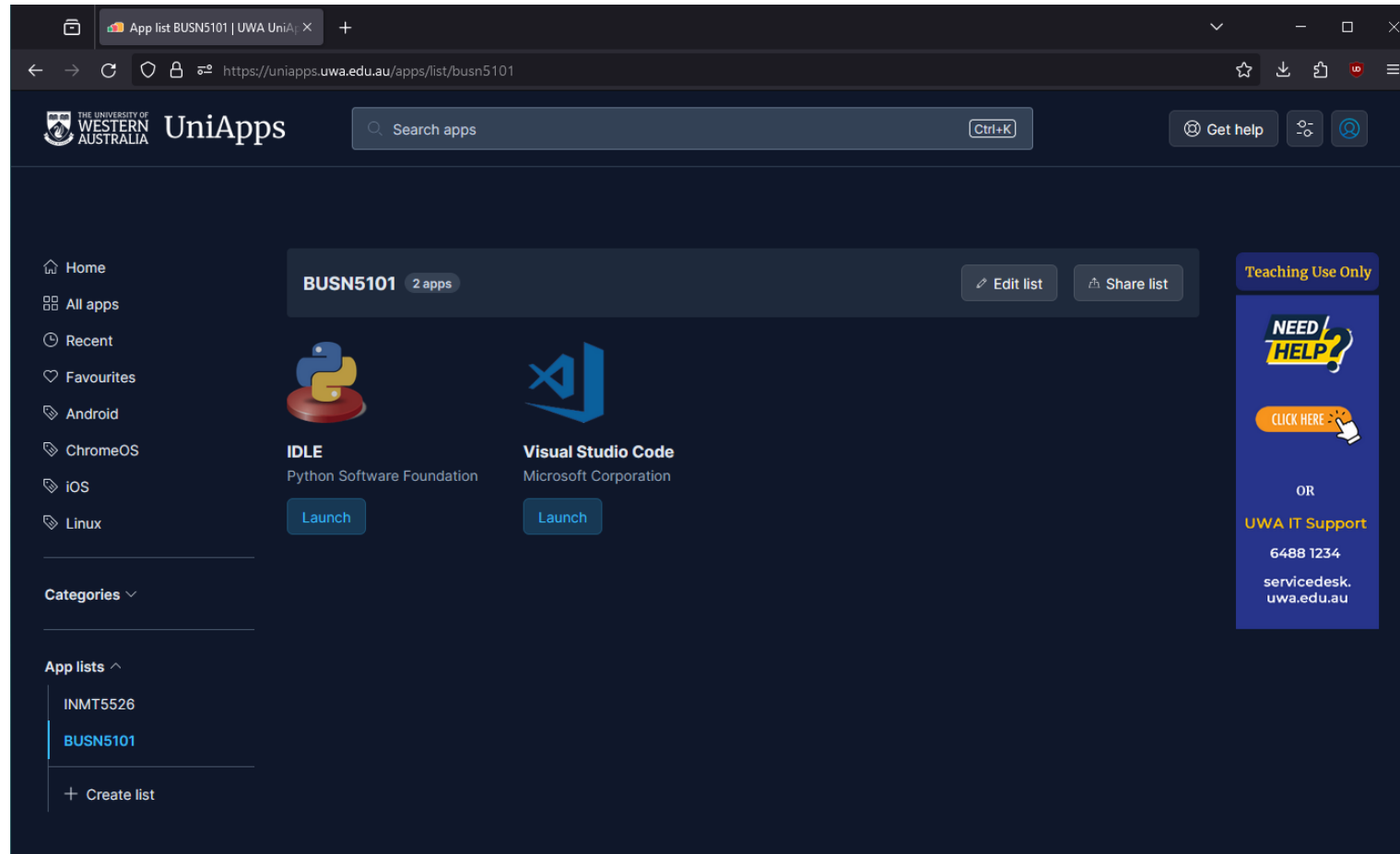
Topic One: Getting Started

BUSN5101: Programming for Business

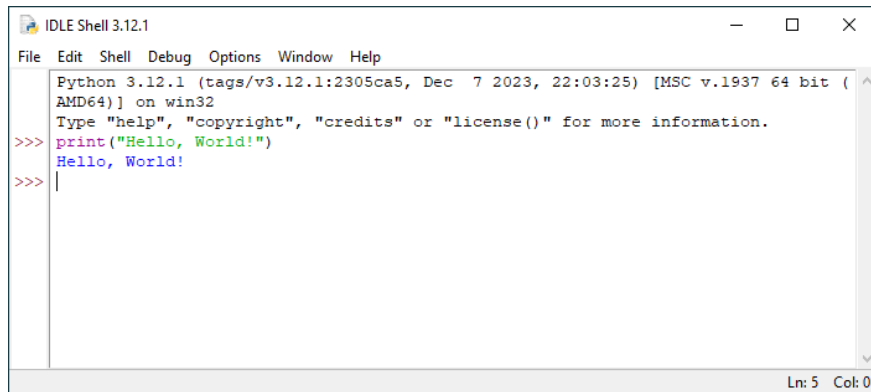
Getting Started with Python

- We will primarily be using the new UniApps system throughout this semester.
 - Alternatively, you can download Python for free from <https://www.python.org/> - it should prompt you to choose the right version for your operating system;
 - We will spend some time in this weeks' lab getting set up – so don't fret;
- It is generally straightforward on most systems (Windows and macOS) to install Python, you will download and run an installer for your system;
 - There are more advanced ways to install Python, including for other systems (e.g. Unix-based). If you are using these systems, including advanced ways for Windows or Mac, then you will know what you are doing;
- Once you have installed Python, you will then need to find a way to use it;
 - The most simple way will be to use the built-in Integrated Development Environment named IDLE.

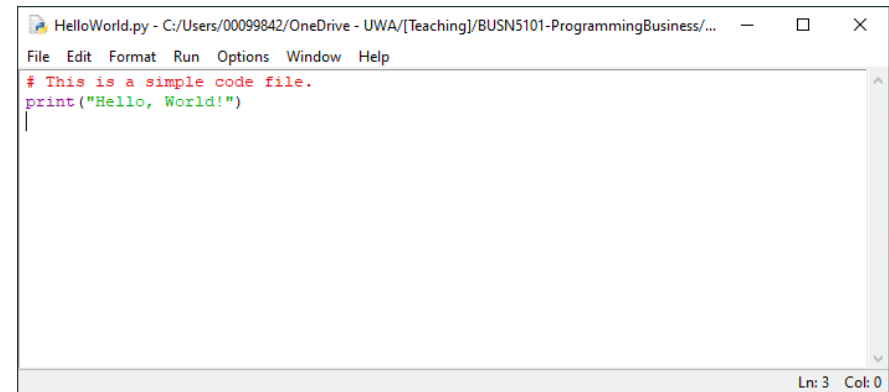
UniApps Interface



- Python's Integrated Development Environment (IDLE) provides the basic tools required to create, execute and debug Python code and is provided with the Python install.
 - This includes auto-complete, syntax highlighting and a formal debugger.
- The program consists of two (types of) windows: the shell window (used for interactive mode) and the editor window (used for script mode).

A screenshot of the IDLE Shell window. The title bar reads "IDLE Shell 3.12.1". The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The text area shows the Python 3.12.1 startup message, followed by a prompt ">>>". The user has entered the command `print("Hello, World!")`, and the output "Hello, World!" is displayed. The status bar at the bottom indicates "Ln: 5 Col: 0".

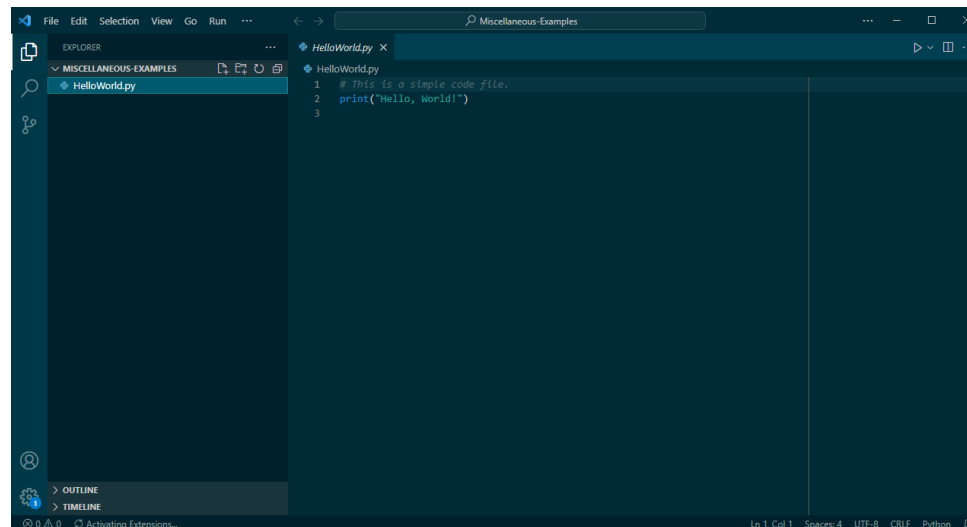
```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hello, World!")
Hello, World!
>>> |
Ln: 5 Col: 0
```

A screenshot of the IDLE editor window. The title bar reads "HelloWorld.py - C:/Users/00099842/OneDrive - UWA/[Teaching]/BUSN5101-ProgrammingBusiness/...". The menu bar includes "File", "Edit", "Format", "Run", "Options", "Window", and "Help". The text area contains a comment `# This is a simple code file.` followed by a line of code `print("Hello, World!")`. The status bar at the bottom indicates "Ln: 3 Col: 0".

```
HelloWorld.py - C:/Users/00099842/OneDrive - UWA/[Teaching]/BUSN5101-ProgrammingBusiness/...
File Edit Format Run Options Window Help
# This is a simple code file.
print("Hello, World!")
|
Ln: 3 Col: 0
```

Other IDE's

- Many other IDE's and Python-related tools exist to provide similar and further tools to help with software development in Python – these exist for other languages too.
 - *Anaconda*: a distribution of Python optimised for data science and machine learning;
 - Plain IDE's include PyCharm, Visual Studio Code, Sublime Text, Notepad++ (debateable).

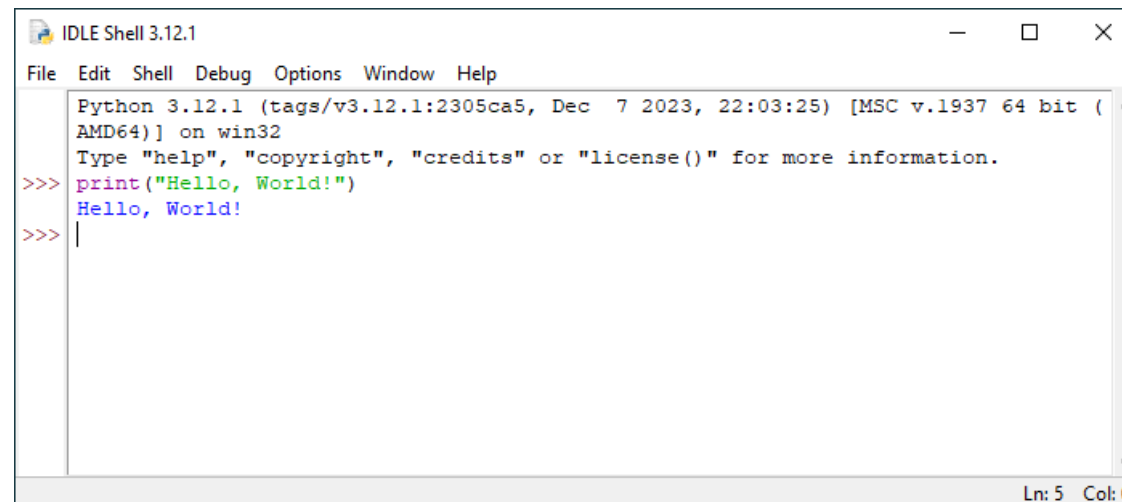


The Most Simple Program

- Traditionally, software developers will create a simple program that outputs the phrase “Hello, World!” to the screen to test out a programming language.
- We can do this in Python using the **print()** command, exactly as follows:
 - `print("Hello, World!")`
- We will go through the process of doing so in this weeks' laboratory;
 - While it may seem simple (and really, it is), there are a few considerations and conventions we should get used to now to make our job easier as we go forward;
 - Some things that are issues in other programming languages aren't in Python (and vice-versa);

Writing in Interactive Mode

- Type the code into the window and press the 'Enter' key to execute the code at once;
- The code `print("Hello, World!")` will print (output) the phrase *Hello, World!* to the screen – changing what is between the quotes will change the output.



```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hello, World!")
Hello, World!
>>> |
```

Ln: 5 Col: 0

Writing in Script Mode

- Script mode is preferred as we can create, read, update and delete source code files, which is helpful for revision, working on things later and code reuse;
 - Need to use *File > New File* or *File > Open* to manipulate the source code files;
 - Type the source code into the window as you would have in Interactive Mode;
 - Then, you will need to use *File > Save As* to ensure the changes are saved (files should have a meaningful name and end in the extension **.py** for Python files);
 - Then, click the *Run* button to execute the source code.
- You will then see the results in the shell window that have been executed from your file.

The End: Thank You

Any Questions? Ask now or via email (tristan.reed@uwa.edu.au)